

Archivist

Investigating Guided Workflows for Reducing Technical Barriers to Local Artificial Intelligence Adoption

Nick Moore | Summer 2026

Abstract

Archivist is a local-first artificial intelligence workspace for schools. Administrators create course workspaces and upload trusted materials; assigned students ask questions and receive locally generated answers with source references. The project investigates whether guided workflows can reduce the technical barriers associated with configuring and deploying local AI assistants for educational use. A reproducible evaluation compared direct generation with retrieval-augmented generation (RAG) across 15 versioned questions, followed by a six-configuration RAG optimization study. RAG increased manually scored correctness from 63.3% to 93.3%. The selected top-k-one configuration achieved 96.7% correctness, 0.97 average groundedness, a 100% retrieval hit rate, and a 1.07-second mean response time. These results validate the technical value of grounding and tuning on the project test set, while the central usability claim still requires stakeholder testing. Future work should evaluate administrator task completion and confidence, then harden OCR, background indexing, security, and storage for broader deployment.

1. Introduction

Educational institutions want the accessibility of conversational AI without surrendering control over course content, student interactions, and institutional data. Hosted assistants lower the barrier to experimentation, but they can create concerns involving privacy, source authority, recurring cost, network dependence, and unsupported answers. Local models address part of that problem by keeping inference and storage under institutional control; however, they introduce a different barrier: setup and maintenance commonly require familiarity with containers, model servers, embeddings, retrieval configuration, and networking.

The central research question is whether guided workflows can significantly reduce the technical barriers associated with configuring and deploying local AI assistants for educational use. Archivist makes this question concrete through an end-to-end minimum viable product (MVP). An administrator can create a workspace, upload approved course sources, add and assign students, and expose a grounded question-answering experience. The system is designed for a school or small organization that values privacy and control but may not have a dedicated AI infrastructure team.

The work pursued three connected objectives. First, it built a usable local application rather than an isolated notebook demonstration. Second, it tested whether retrieval improves source-specific answers over direct local generation. Third, it optimized retrieval settings to identify a practical quality–latency balance. The broader research objective—whether guided deployment reduces human effort and errors—remains the next empirical phase.

Technically, Archivist uses a Go HTTP server, server-rendered HTML with partial updates, SQLite storage, and Ollama for local chat and embedding inference. The application separates interface handlers from shared services. Uploaded files are extracted, divided into overlapping chunks, embedded, and stored with pipeline-version metadata. For each student question, the retrieval service embeds the query, ranks stored chunks using cosine similarity, supplies selected evidence to a context-constrained prompt, and returns an answer with source references. Docker Compose packages the application and model service into a repeatable local deployment.

DESIGN PRINCIPLE

Local by default Course documents, embeddings, model calls, accounts, and chat history remain under the operator's control.

2. Methodology

Development followed an iterative workflow: define the stakeholder path, implement the smallest complete vertical slice, test it locally, document failure modes, and convert those failures into regression checks. The resulting path begins with first-run administration and continues through workspace creation, source ingestion, assignment, retrieval, and cited response generation. The web interface and terminal interface share application services so business rules are not duplicated in templates or handlers.

For the baseline experiment, the evaluation used 15 versioned questions about a bundled, instructor-prepared paraphrase of an OpenStax data-science section. The questions covered definitions, comparisons, recall, reasoning, synthesis, and two deliberately unanswerable items. Both conditions used the same local chat model and deterministic generation settings. The no-RAG condition received only the question. The RAG condition additionally received the three most similar source chunks, using 1,000-character chunks, 200-character overlap, nomic-embed-text embeddings, and cosine similarity.

Each run captured responses, retrieved chunk identifiers, similarity scores, retrieval time, generation time, and total response time. Manual review scored correctness on a 0–2 rubric: 2 for correct and complete, 1 for partially correct or incomplete, and 0 for incorrect or missing. RAG answers were also scored for groundedness against the retrieved text. Retrieval hit rate measured whether retrieved context contained the evidence required by the reference answer, and hallucination handling measured whether unanswerable questions avoided unsupported content.

The optimization experiment evaluated six RAG configurations using the same 15 questions. The study varied chunk size and overlap—600/100, 800/150, 1,000/200, and 1,400/250 characters—and varied top-k at 1, 3, and 5 for the 1,000/200 setting. A project-specific selection score combined correctness, groundedness, retrieval success, and response time. The score was used for engineering selection rather than as a general benchmark.

To support reproducibility, the dataset, source text, experiment settings, run metadata, raw answers, manual-review files, and summaries were retained in the repository. Source and configuration hashes support comparison across runs. The design intentionally favors auditability, but it does not provide statistical significance: the question set is small, the source is narrow, and the manual rubric is reviewer-dependent.

3. Results

On the final baseline evaluation, direct generation achieved 63.3% of the maximum correctness score. Adding retrieved course context increased correctness to 93.3%, a gain of 30.0 percentage points. Retrieval Hit Rate@3 was 100%, average groundedness was 0.93 on a 0–1 scale, and hallucination handling on the questions marked unanswerable from the section was 100%. Mean response time increased modestly from 1.11 seconds without RAG to 1.25 seconds with RAG.

Metric	No RAG	RAG	Difference
Correctness	63.3%	93.3%	+30.0 pts
Mean response time	1.11 s	1.25 s	+0.14 s
Retrieval Hit Rate@3	—	100%	—
Average groundedness	—	0.93	—
Hallucination handling	—	100%	—

Table 1. Final baseline comparison across 15 manually scored questions.

The largest difference appeared when questions required source-specific wording or boundaries. Direct generation could produce plausible general knowledge while missing the assigned material’s intended explanation. On an out-of-scope question about transformer embeddings, the no-RAG answer invented a technically plausible explanation even though the section did not contain one; RAG correctly declined to answer from the available material. This behavior matters in education because a fluent answer can still conflict with the instructor-approved source.

Retrieval success did not guarantee complete generation. Some answers used the correct evidence but omitted a required qualification or overstated the source. This distinction justified separate retrieval-hit, groundedness, and correctness measures rather than collapsing all behavior into one pass rate.

KEY RESULT

+30.0 points

Grounding improved correctness far more than the observed 0.14-second mean latency cost on this test set.

All six configurations achieved a 100% retrieval hit rate and 100% hallucination handling on the small test set. Correctness ranged from 93.3% to 100%, while mean response time ranged from 1.07 to 1.88 seconds. The top-k-one configuration used 1,000-character chunks with 200-character overlap and retrieved one chunk. It achieved 96.7% correctness, 0.97 groundedness, and a 1.07-second mean response time. Relative to the top-k-three baseline, correctness increased 3.3 points and mean latency fell 0.81 seconds, or 42.9%.

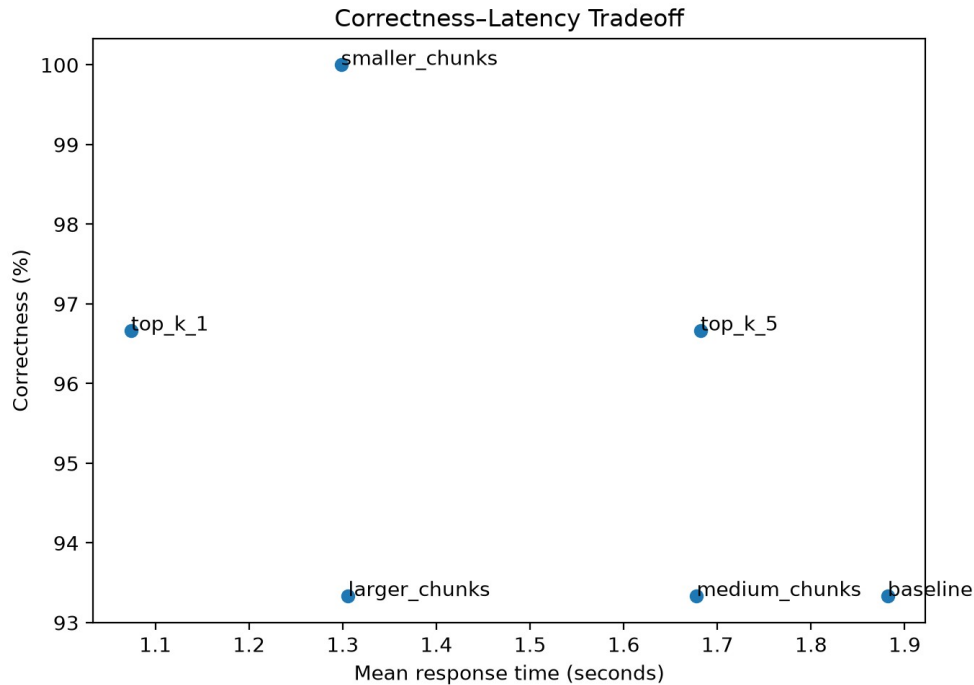


Figure 1. Correctness-latency tradeoff across six RAG configurations.

4. Challenges and Responses

An early evaluation run returned HTTP 404 errors for every generation request while embedding and retrieval continued. The artifacts therefore showed near-zero latency and zero quality—numbers that could have been mistaken for poor model performance. The evaluator was changed to exit unsuccessfully when model requests fail and to preserve failed-run artifacts for diagnosis without treating them as quality evidence.

A second challenge was that more context was not consistently better. Increasing top-k expanded the prompt and slowed generation without improving retrieval hits. Conversely, top-k one preserved strong answer quality while reducing latency and retrieved context. The response was to evaluate configurations with a combined quality-speed score and retain the full metric set so the selection remained auditable.

5. Discussion

The technical evidence supports retrieval-augmented generation for source-specific educational assistance. Grounding substantially improved correctness and boundary handling, and the selected configuration showed that a small local model can respond in roughly one second on the evaluated hardware. These findings strengthen the case for a controlled assistant that prioritizes instructor-approved material over unconstrained general knowledge.

The results also show why system quality cannot be represented by a single accuracy number. A deployment can fail because the model service is unavailable, retrieval misses the necessary evidence, generation omits part of the evidence, or an answer adds unsupported information. Archivist's evaluation therefore separates operational availability, retrieval, correctness, groundedness, hallucination handling, and latency. This decomposition makes debugging more useful and reduces the risk of interpreting infrastructure errors as model behavior.

However, the project has not yet demonstrated that guided workflows significantly reduce setup difficulty for real administrators. The MVP makes that study possible, but the current experiments measure answer behavior rather than human task performance. The 15-question dataset is too small for broad generalization, the questions were researcher-written, the source covers one short section, and one manual reviewer scored the final baseline. The optimization quality scores are an AI-assisted draft pending final human verification. No statistical significance is claimed.

The application also retains engineering limitations. The current extraction path supports text-based PDFs but not scanned or image-only documents. Embeddings are stored as JSON in SQLite and scanned in process, favoring inspectability over large-scale retrieval. Reindexing has route and interface foundations but not a fully observable background-job worker. Chat history is grouped by workspace rather than named conversations. Production hardening also requires CSRF protection, rate limiting, production TLS, richer account administration, and broader operational monitoring.

Finally, local deployment reduces dependence on hosted APIs but does not eliminate governance responsibilities. Schools still need clear policies for approved sources, account access, retention, model updates, and review of high-impact answers. Citations help students and instructors verify responses, but they should support—not replace—source reading and instructional judgment. The system should communicate uncertainty and refuse unsupported questions rather than presenting fluent speculation as course knowledge.

6. Future Work

The immediate next phase is a stakeholder usability study. Participants should complete first-run setup, create a workspace, upload a source, create and assign a student, and obtain a cited answer. Measures should include task completion rate, time on task, setup errors, recovery behavior, requests for assistance, and perceived confidence before and after the workflow. A comparison against an unguided setup path would directly test the primary research question.

The evidence base should also expand to include a larger independently authored and held-out question set, multiple source types and academic domains, repeated runs across seeds, multiple human reviewers, and inter-rater agreement. Targeted experiments should evaluate token-aware chunking, reranking, metadata filters, alternative embeddings, larger local chat models, and answer-level citation precision. Regression cases should preserve every observed retrieval, generation, and availability failure.

In parallel, product work should add OCR and layout-aware extraction, observable background ingestion and reindexing jobs, named conversation sessions, richer source previews, model-health guidance, secure password administration, CSRF protection, rate limiting, TLS, and scalable vector storage. Offline Linux deployment should continue to verify network isolation and provide reversible, plain-language installation guidance.

7. Conclusion

Archivist demonstrates a credible technical path for private, source-grounded educational AI. Retrieval raised correctness by 30 percentage points in the final baseline, while configuration tuning produced a faster selected system with 96.7% correctness and strong groundedness. The project's most important contribution is the integration of local inference, trusted-source retrieval, guided administration, reproducible evaluation, and explicit failure analysis into one deployable MVP. The next milestone is not another isolated model score; it is evidence that real administrators can successfully complete the workflow with fewer errors and greater confidence.

References and Project Artifacts

- [1] Archivist repository, README.md and docs/capstone-notes.md, project scope, architecture, limitations, and roadmap.
- [2] results/baseline_results.csv and results/report_summary.txt, final 15-question no-RAG versus RAG manual evaluation.
- [3] results/optimization_summary.csv, results/optimization_metadata.json, and results/optimization_report_summary.txt, six-configuration RAG study.
- [4] evaluation/README.md and scripts/evaluate.py, versioned evaluation protocol and reproducible runner.
- [5] docs/architecture.md, docs/setup.md, and docs/linux-deployment.md, implementation and deployment design.